INT 428 · PROJECT VIVA PREPARATION GUIDE

SafetyNet.ai

AI-Powered Emergency Fund & Financial Risk Analyser — Built for India

React 19 | Express 5 | Node.js | LLaMA-3.3-70b (Groq) | XGBoost | Python / Flask | MongoDB + Mongoose | JWT Auth | Tailwind v4 | Framer Motion | jsPDF | Nodemailer

Prepared: April 29, 2026 | Full-Stack AI Web Application

🖨️ Save as PDF

🎯 1. Project Overview — What is SafetyNet.ai?

SECTION 1

SafetyNet.ai is a full-stack, AI-powered web application that helps Indian individuals assess their **financial risk** and calculate their ideal **emergency fund**. It combines a rule-based financial engine, a Large Language Model (LLaMA via Groq), and a trained Machine Learning model (XGBoost) to give personalised, data-driven financial guidance.

CORE PROBLEM

Why does this exist?

Most Indians don't have an emergency fund. Sudden job loss, medical emergency, or financial shock can be devastating. This tool helps people understand their risk and build a safety net.

SOLUTION

What does it do?

Takes 14 financial inputs (income, EMI, savings, job type, city, age etc.), runs 3 parallel analysis engines, and outputs a risk score (0–100), recommended fund size, and actionable steps.

TARGET USERS

Who uses it?

Indian working professionals across Tier 1/2/3 cities — salaried employees, freelancers, gig workers, business owners, and government employees who want personalised financial risk assessment.

💡 **Key Innovation:** The project combines three independent analysis engines — a rule-based financial algorithm, a Groq LLM (LLaMA-3.3-70b), and a trained XGBoost ML model — and shows an *AI/ML agreement indicator* to build user trust.

2. System Architecture

SECTION 2

Three-Tier Architecture

Frontend (Client)

- React 19 + Vite 8
- Tailwind CSS v4
- Framer Motion animations
- Recharts for data viz
- jsPDF for report export
- Web Speech API (voice)
- Runs on port 5173

Backend (Server)

- Express 5 + Node.js
- REST API (JSON)
- JWT authentication
- bcryptjs password hashing
- Nodemailer (email)
- MongoDB + Mongoose ORM
- Runs on port 5001

ML Service (Python)

- Flask REST API
- XGBoost models
- scikit-learn pipeline
- Trained on 8,000 samples
- R^2 = 0.949, Acc = 85.3%
- Runs on port 5002

Request Flow (POST /api/analyze)

User fills form → React sends POST → Express validates → Calc Engine ⇒

Groq LLM parallel with XGBoost ML → Unified JSON → Dashboard

The Groq AI call and the XGBoost ML prediction run **in parallel** using `Promise.all()`, so the response time is determined by whichever is slower (max ~8s for AI, ~4s for ML), not the sum of both.

Key Design Decisions

MongoDB + File Fallback

If `MONGO_URI` is not set, the app uses a local `data.json` file for storage — making it runnable without a database for demos.

ML Microservice Pattern

The XGBoost model runs as a separate Flask microservice on port 5002. If it's offline, Node.js gracefully falls back and returns `ml: null`.

LLM Fallback Engine

CORS Security

If the Groq API key is missing or the API fails, a high-quality *rule-based insight engine* generates insights — the app never breaks.

The backend only allows origins listed in `CLIENT_ORIGIN` env variable. The JSON body is capped at 10KB to prevent payload attacks.

3. Technology Stack — Detailed Explanation

SECTION 3

LAYER	TECHNOLOGY	VERSION	WHY THIS CHOICE?
Frontend	**React 19**	19.2.5	Component-based UI, hooks for state management, Context API for theme/auth
Build Tool	**Vite 8**	8.0.10	Lightning-fast HMR, ESM-native, much faster than CRA/Webpack
Styling	**Tailwind CSS v4**	4.2.4	Utility-first CSS, no unused styles, custom design tokens via CSS variables
Animation	**Framer Motion**	12.38	Declarative animations, AnimatePresence for page transitions
Charts	**Recharts**	3.8.1	React-native chart library, AreaChart + PieChart for financial projections
PDF Export	**jsPDF**	4.2.1	Client-side PDF generation — no server needed for report download
Backend	**Express 5**	5.2.1	Minimal, fast Node.js framework; v5 has async error handling improvements
Database ORM	**Mongoose**	9.5.0	Schema validation, virtuals, indexes for MongoDB
Auth	**JWT + bcryptjs**	9.0.3 / 3.0.3	Stateless auth — no session storage needed; bcrypt cost factor 12 for passwords
HTTP Logging	**Morgan**	1.10.1	Standard HTTP request logger piped into custom Winston/logger
AI/LLM	**Groq + LLaMA-3.3-70b**	—	Fastest LLM inference available (~500 tokens/sec); free tier sufficient for demos
ML Model	**XGBoost**	—	Gradient boosting excels at tabular data; trained on 8,000 synthetic profiles
ML Server	**Flask**	—	Lightweight Python web server; perfect for serving ML model predictions

LAYER	TECHNOLOGY	VERSION	WHY THIS CHOICE?
Email	**Nodemailer**	8.0.7	Welcome email on registration via Gmail SMTP

4. Financial Calculation Engine — The Core Algorithm SECTION 4

File: `server/services/calculationService.js` — This is the heart of the application. It takes 14 inputs and produces a **Risk Score (0–100)** and a **Recommended Emergency Fund** amount.

Step 1: Base Calculation

> **Base Fund = (Monthly Expenses + EMI) × 3 months minimum**
> The minimum recommended emergency fund is 3 months of total obligations.

Step 2: The 9 Risk Factors (14 Input Dimensions)

#	FACTOR	RISK SCORE IMPACT	EXTRA MONTHS ADDED	EXPLANATION
3a	**Employment Type**	Govt: +5, Corporate: +22, Business: +38, Gig: +42, Freelancer: +48	0 to 4.5 months	Freelancers have irregular income — need 4.5 extra months buffer
3b	**Dependents**	+7 per dependent	+0.8 per dependent	Each family member increases financial obligation
3c	**EMI/Debt Load**	>50%: +32, 30–50%: +16, <30%: +6	+0.5 to +2.5	RBI recommends keeping EMI below 30% of income
3d	**City Tier**	Tier 1: +8	Tier 1: +1, Tier 3: −0.5	Tier 1 metro has 18% higher cost of living multiplier
3e	**Age**	>50: +10, <27: −5	>50: +1 month	Pre-retirement phase needs larger buffer
3f	**Health Insurance**	No insurance: +18	+1 month if uninsured	Medical bills ₹2–10L are the #1 cause of financial ruin in India
3g	**Rent vs Own**	Renting: +5	0	Rent is a fixed obligation during income loss
3h	**Savings Rate**	>85% spending: +20, <55% spending: − (protective)	0	Spending more than 85% of income leaves no buffer

#	FACTOR	RISK SCORE IMPACT	EXTRA MONTHS ADDED	EXPLANATION
3i	**Life Stage**	With kids/single parent: +8	+1 month	Family responsibilities increase financial risk

Step 3: Risk Level Classification

SCORE RANGE	RISK LEVEL	COLOR
0 – 25	**Low Risk**	Emerald Green
26 – 50	**Medium Risk**	Amber
51 – 72	**High Risk**	Orange
73 – 100	**Critical Risk**	Red

City Cost Multiplier

The final fund amount is multiplied by the city tier cost factor:

- **Tier 1 Metro:** ×1.18 (18% more expensive)
- **Tier 2 City:** ×1.0 (baseline)
- **Tier 3 Town:** ×0.82 (18% cheaper)

Step 4: Investment Tier Split (3-Tier Strategy)

TIER 1 — LIQUID

Savings Account / Cash

1 month of expenses. Instantly accessible. For immediate emergencies (hospital admission, same-day need).

TIER 2 — LIQUID MF

Liquid Mutual Funds

2 months of expenses. T+1 redemption. Better returns than savings (~7% p.a.). For 1–2 week emergencies.

TIER 3 — FD

Fixed Deposits

Remaining fund. Higher interest rate (~7.5% p.a.). For longer-term job loss scenarios (2–6 months).

Output Fields Returned

```
// Key output fields from calculationService.calculateEmergencyFund()
{
  riskScore:          62,          // 0-100
  riskLevel:          "High",       // Low / Medium / High / Critical
  recommendedFund:    540000,      // ₹5.4 Lakh target fund
  monthsRecommended:  7.5,          // 7.5 months buffer recommended
  savingsGap:         290000,      // ₹2.9L still to be saved
```

```
  percentFunded:      46,           // 46% of goal achieved
  monthsCovered:      3.2,          // current savings cover 3.2 months
  survivalMonths:     3.2,          // same as monthsCovered
  surplusIncome:      15000,        // monthly surplus available to save
  suggestedMonthly:   8000,         // recommended monthly savings amount
  riskFactors:        [...],        // array of risk factor objects
  protectiveFactors:  [...],        // array of positive factors
  tierPlan:           {...},        // 3-tier investment breakdown
  projection:         [...],        // 12-month savings projection array
}
```

5. Machine Learning Model — XGBoost

SECTION 5

What is XGBoost?

XGBoost (Extreme Gradient Boosting) is an ensemble ML algorithm that builds multiple decision trees sequentially, where each tree corrects the errors of the previous one. It's the go-to algorithm for structured/tabular data and has won numerous Kaggle competitions.

Why XGBoost for this project?

- Financial data is tabular (rows & columns) — XGBoost excels here
- Handles missing values internally
- Provides feature importance scores
- Fast training and inference
- No need for feature scaling (unlike neural networks)

Model Performance Metrics

- **Training Samples:** 6,400 (80% of 8,000)
- **Test Samples:** 1,600 (20% of 8,000)
- **Regressor R^2:** 0.949 (94.9% variance explained)
- **Classifier Accuracy:** 85.3%
- **MAE (Mean Absolute Error):** 4.22 risk score points

Training Data Generation (train_model.py)

⚠ The model is trained on **synthetically generated data** — 8,000 Indian household profiles created programmatically using realistic distributions. This is valid because the labels are computed using the exact same rule-based algorithm as `calculationService.js`, so the ML model learns to approximate the financial logic.

The synthetic data generation uses:

- Income: ₹15,000–₹3,00,000/month (log-normal distribution)
- Expenses: 40–80% of income
- EMI: 0–50% of income
- Job types distributed: 20% govt, 40% corporate, 15% freelancer, 15% business, 10% gig

The 16 Features Used

Raw Input Features (8)

FEATURE	DESCRIPTION
`income`	Monthly income in ₹

Derived / Encoded Features (8)

FEATURE	HOW COMPUTED

`expenses`	Monthly expenses in ₹
`emi`	Monthly EMI obligations in ₹
`savings`	Current savings in ₹
`dependents`	Number of dependents (0–10)
`age`	Age in years (18–90)
`has_insurance`	Binary: 1=insured, 0=uninsured
`renting`	Binary: 1=renting, 0=owns

`job_enc`	Label-encoded job type (0–4)
`life_enc`	Label-encoded life stage (0–4)
`city_enc`	Label-encoded city tier (0–2)
`emi_ratio`	EMI ÷ Income
`expense_ratio`	Expenses ÷ Income
`surplus`	Income − Expenses − EMI
`savings_ratio`	Savings ÷ (Income × 12)
`survival_months`	Savings ÷ Expenses

Dual Model Architecture

MODEL 1: REGRESSOR

XGBoostRegressor — Predicts Risk Score

Outputs a continuous value 0–100. **n_estimators=400, max_depth=7**. Evaluates R^2 and MAE on test set.

MODEL 2: CLASSIFIER

XGBoostClassifier — Predicts Risk Level

Outputs Low/Medium/High/Critical + confidence probabilities. Same hyperparameters. Evaluates accuracy score.

Flask Service (serve.py) — API Endpoints

ENDPOINT	METHOD	DESCRIPTION
`/predict`	POST	Takes financial inputs, returns mlRiskScore, mlRiskLevel, confidence, probabilities
`/health`	GET	Returns service status + model metadata (R^2, accuracy)
`/feature-importance`	GET	Returns XGBoost feature importance scores

🧠 6. AI Integration — Groq + LLaMA-3.3-70b

SECTION 6

What is Groq?

Groq is an AI inference platform that runs large language models at extremely high speed (~500 tokens/second) using custom LPU (Language Processing Unit) hardware. It provides an OpenAI-compatible API.

LLM Used: LLaMA-3.3-70b-versatile

Meta's open-source large language model with 70 billion parameters. Used for the chatbot endpoint. The analysis endpoint uses `llama-3.1-8b-instant` for faster response.

API Parameters Used

- **temperature: 0.35** — low = more deterministic, factual financial advice
- **max_tokens: 600** — controlled output length
- **response_format: json_object** — structured output
- **timeout: 8000ms** — fail fast, use fallback

What does the AI generate? (aiService.js)

```
// Groq returns structured JSON:
{
  "insights":    "Personalised paragraph about the user's financial situation",
  "suggestions": ["Action step 1", "Action step 2", ...],  // 5 action items
  "warnings":    ["Warning about health insurance gap", ...]     // specific risks
}
```

Fallback Engine

✅ If the Groq API is unavailable (no key, rate limit, network error), the **rule-based fallback engine** in `aiService.js` generates insights based on the risk level, EMI ratio, insurance status, and job type. Users always get a useful response.

Chatbot System Prompt Design (chatController.js)

The chatbot is given a detailed system prompt that:

- Defines expertise areas: FDs, Liquid MFs, PPF, EPF, NPS, ELSS, SIPs
- Instructs to use ₹ (INR) for all amounts
- Limits answers to 180 words unless asked for detail

- Injects user's financial profile context if available (risk score, income, EMI)
- Includes SEBI disclaimer (educational info, not registered financial advice)
- Keeps last 8 conversation turns for context

Voice Features (ChatBot.jsx)

Voice Input — Web Speech API

Uses browser's `SpeechRecognition` with `lang='en-IN'` for Indian English accent recognition. Supports continuous speech-to-text.

Voice Output — SpeechSynthesis

Uses browser's `speechSynthesis.speak()` to auto-read AI responses aloud. Shows animated sound-wave while speaking. Each bubble has play/stop controls.

7. Authentication System — JWT + bcrypt

SECTION 7

Registration Flow (POST /api/auth/register)

Input validation → Email uniqueness check → bcrypt.hash(pass, 12) →
Save to users.json → Send welcome email → Return JWT token

Login Flow (POST /api/auth/login)

Find user by email → bcrypt.compare() → Return JWT token

Security Details

Password Hashing (bcryptjs)

- **Cost factor: 12** — industry standard (2^12 iterations)
- Each password has a random salt automatically
- Even if database leaks, passwords cannot be reverse-engineered
- Never stored in plain text

JWT Token

- **Expiry: 7 days**
- Payload: `{ id, email, name, role }`
- Verified on every protected request via `auth.js` middleware
- Stored in `localStorage` on the client

Role-Based Access Control

- Two roles: `user` and `admin`
- **First registered user becomes admin**
- Admin routes protected by `requireAdmin` middleware
- Regular users cannot access `/api/admin/*`

Input Validation

- Email format validated with regex
- Password minimum 6 characters
- Name, email, password all required
- Email stored in lowercase

JWT Middleware (auth.js)

```
const verifyToken = (req, res, next) => {
  const header = req.headers['authorization'];
```

```
  const token  = header?.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'Authentication required' });
  try {
    req.user = jwt.verify(token, JWT_SECRET);  // decode & attach to request
    next();
  } catch {
    return res.status(403).json({ error: 'Invalid or expired token' });
  }
};
```

8. Database Design — MongoDB + Mongoose

SECTION 8

Dual Storage Strategy

PRIMARY: MONGODB

When MONGO_URI is configured

Full Mongoose ORM, schema validation, indexes, virtuals, connection pooling (max 10 connections), auto-reconnect with event listeners.

FALLBACK: JSON FILES

When no DB configured

`data.json` for analysis records (max 100), `users.json` for user accounts. Zero external dependency — works anywhere.

UserData Mongoose Schema

```
const UserDataSchema = new Schema({
  inputs:     InputsSchema,       // income, expenses, emi, savings, jobType, etc.
  results:    Schema.Types.Mixed, // full calculation blob (flexible)
  aiInsights: AiInsightsSchema,   // { insights, suggestions[], warnings[], source }
  savedAt:    { type: Date, default: Date.now, index: true },
}, { timestamps: true, collection: 'userdata' });

// Indexes for performance:
UserDataSchema.index({ savedAt: -1 });                    // history queries (newest firs
UserDataSchema.index({ 'inputs.jobType': 1, savedAt: -1 }); // analytics by job type
```

MongoDB Connection Config (db.js)

- **serverSelectionTimeoutMS: 5000** — fails fast if MongoDB unreachable
- **socketTimeoutMS: 45000** — closes idle sockets after 45s
- **maxPoolSize: 10** — max 10 concurrent connections
- Connection lifecycle events: `disconnected`, `reconnected`, `error`

9. Frontend Components — React Architecture

SECTION 9

Application State Management (App.jsx)

The root `App.jsx` uses React hooks and Context API — no Redux or external state library needed:

State Variables

- `view` — current page (landing, portal, dashboard, history, auth, admin)
- `results` — analysis result from API
- `dark` — theme toggle (dark/light)
- `user` — logged-in user object (from localStorage)
- `token` — JWT token (from localStorage)

Context Provided

`ThemeCtx` — provides `{ dark, toggle }` to all child components. Any component can call `useTheme()` hook to read the current theme and toggle it.

Auth state is passed as props down to components that need user info or the JWT token for API calls.

Page / Component Map

COMPONENT	ROUTE/VIEW	PURPOSE
`Landing.jsx`	landing	Home page — hero section, feature list, CTA buttons
`AuthPage.jsx`	auth	Sign In / Register toggle — calls POST /api/auth/login or /register
`PortalPage.jsx`	portal	Logged-in dashboard — quick start, recent history, user greeting
`CalculatorForm.jsx`	calculator	6-step multi-step form with sliders, tooltips, review step
`AnalysisPage.jsx`	analysis	Boot animation sequence while API call is made
`Dashboard.jsx`	dashboard	5-tab results dashboard — Overview, AI Insights, Projection, Stress Test, ML
`HistoryPanel.jsx`	history	List of saved analyses — calls GET /api/history
`ChatBot.jsx`	global (all pages)	Floating chatbot with voice input/output, keyboard shortcut Ctrl+K
`AdminPage.jsx`	admin	User management, system stats, risk distribution chart

Dashboard — 5 Tabs Explained

TAB 1: OVERVIEW

Key Metrics + Risk Gauge

Animated SVG risk gauge (0–100 arc), metric cards for fund target, savings gap, months covered. Risk factors and protective factors listed.

TAB 2: AI INSIGHTS

LLM-Generated Advice

Groq insights paragraph, 5 action suggestions, specific warnings. Shows AI/ML agreement indicator. Smart suggestions banner.

TAB 3: PROJECTION

12-Month Chart

Recharts AreaChart showing monthly savings growth towards the target. Interactive slider to adjust monthly saving amount. Target reached date shown.

TAB 4: STRESS TEST

Income Drop Simulation

Simulates income drop by X% and expense spike by Y%. Recalculates how many months the fund covers under worst-case scenarios.

TAB 5: ML ANALYSIS

XGBoost Prediction

Shows XGBoost's independent risk score and level. Confidence percentage. Probability bars for each risk level. Agreement/disagreement indicator with rule-based model.

3-TIER PLAN

Investment Breakdown

Pie chart + breakdown of how to split the emergency fund across Savings Account, Liquid MF, and Fixed Deposit.

10. All API Endpoints — Complete Reference

SECTION 10

METHOD	ENDPOINT	AUTH?	DESCRIPTION
POST	`/api/analyze`	No	Full analysis — calc engine + Groq AI + XGBoost ML (primary endpoint)
POST	`/api/calculate`	No	Fast calculation only — no AI, instant response (backward-compatible alias)
POST	`/api/save`	Optional	Save an analysis result to history
POST	`/api/history/save`	Optional	Legacy alias for /api/save
GET	`/api/history`	Optional	Paginated list of saved analysis records
GET	`/api/health`	No	Server health check — DB status, version, uptime
POST	`/api/auth/register`	No	Register new user — returns JWT token
POST	`/api/auth/login`	No	Login — returns JWT token
GET	`/api/auth/me`	JWT Required	Get current logged-in user details
POST	`/api/chat`	No	AI chatbot — sends message, gets Groq response
GET	`/api/admin/stats`	Admin JWT	Total users, analyses, risk distribution, avg fund target
GET	`/api/admin/users`	Admin JWT	List all registered users
POST	`/api/admin/users/:id/role`	Admin JWT	Change a user's role

▸ **POST /api/analyze — Input Schema**

```
{
  "monthlyIncome":      60000,         // Required — ₹ per month
  "monthlyExpenses":    30000,         // Required — ₹ per month
  "emi":                12000,         // Optional — loan EMI ₹/month
  "savings":            80000,         // Optional — current total savings ₹
  "jobType":            "corporate",   // govt | corporate | freelancer | business | gig
  "dependents":         2,             // 0–10
  "cityTier":           "1",           // "1" | "2" | "3"
  "age":                32,            // 18–90
  "lifeStage":          "mid_career", // single | mid_career | married_with_kids | pre_retir
  "hasHealthInsurance": "yes",         // "yes" | "partial" | "no"
  "rentOrOwn":          "rent"         // "rent" | "own_loan" | "own_free"
}
```

11. Security Features (OWASP Alignment)

SECTION 11

A01: BROKEN ACCESS CONTROL

JWT middleware on all protected routes. Role-based access control (user vs admin). Admin routes double-checked with `requireAdmin` guard.

A02: CRYPTOGRAPHIC FAILURES

Passwords hashed with bcrypt (cost 12). JWT signed with secret key. HTTPS recommended for production (TLS).

A03: INJECTION

All inputs validated and sanitised before use. No raw SQL. MongoDB queries use Mongoose schema which prevents NoSQL injection.

A05: SECURITY MISCONFIGURATION

CORS restricted to known origins. JSON body size capped at 10KB. Environment variables for all secrets (not hardcoded).

A07: AUTH FAILURES

Timing-safe password comparison via bcrypt.compare. Token expiry (7 days). Email uniqueness enforced on registration.

A09: LOGGING FAILURES

Structured logging (morgan + custom logger). All auth events, API errors, and AI/ML calls are logged with context for audit trail.

? 12. Expected Viva Questions & Model Answers

SECTION 12

Basic / Project Understanding

Q1. What is the main objective of your project?

SafetyNet.ai helps Indian individuals understand their financial risk and calculate the ideal emergency fund they need. It solves the real problem that most Indians are financially unprepared for emergencies — job loss, medical bills, or sudden expenses. It uses 14 financial factors, an AI (LLaMA LLM), and a trained ML model (XGBoost) to provide personalised, data-driven recommendations.

Q2. Why did you choose a full-stack approach with separate services?

The application has three distinct concerns: a real-time interactive UI (React), complex business logic and auth (Node/Express), and a computationally heavy ML model (Python/XGBoost). Separating them as microservices allows each to scale independently, use the best language for the task (Python for ML), and fail gracefully without bringing down the whole application.

Q3. How does your system handle failures — what if Groq or the ML service is down?

The system has three layers of fallback: (1) If XGBoost Flask service is offline, the Node server catches the ECONNREFUSED error and returns ml: null — the analysis still works with just the rule-based engine. (2) If Groq API fails, the aiService falls back to a high-quality rule-based insight engine. (3) If MongoDB is not configured, the system uses JSON files for storage. The user always gets a complete, useful response.

Algorithm & Calculation

Q4. How do you calculate the risk score? Walk me through the algorithm.

The risk score is a weighted composite of 9 factors. Starting from 0, I add: job type score (5 for govt, up to 48 for freelancer), 7 points per dependent, 6–32 points based on EMI-to-income ratio (RBI recommends below 30%), 8 for Tier-1 city, 10 for age above 50 (minus 5 for young age), 18 for no health insurance, 5 for renting, and 8–20 based on savings/expense ratio. The total is clamped to 0–100 and mapped to Low/Medium/High/Critical buckets.

Q5. Why is "no health insurance" the biggest single risk factor (+18 points)?

Medical emergencies are the #1 cause of financial ruin in India. A single hospitalisation can cost ₹2–10 lakhs — which would wipe out most people's emergency funds. Without insurance, the

emergency fund must also absorb healthcare costs, requiring a significantly larger buffer. That's why we add an extra month to the recommendation and 18 risk points.

Q6. What is the 3-tier emergency fund strategy?

Following Indian financial planning best practices: Tier 1 is 1 month in a savings account (instantly accessible for same-day emergencies), Tier 2 is 2 months in liquid mutual funds (T+1 redemption, ~7% returns), and Tier 3 is the remaining amount in Fixed Deposits (~7.5% returns, for longer job-loss scenarios of 2–6 months). This balances accessibility with returns.

Machine Learning

Q7. Why did you use XGBoost instead of a neural network or linear regression?

XGBoost is the industry standard for tabular/structured data like financial profiles. Neural networks need large datasets and extensive hyperparameter tuning for tabular data. Linear regression cannot capture non-linear relationships (e.g., EMI ratio has a stepped effect, not linear). XGBoost handles mixed feature types, missing values, and provides feature importance — perfect for our use case. Our R^2 of 0.949 on test data validates this choice.

Q8. Your training data is synthetic — isn't that a problem?

Not in this case. The labels (risk scores) are generated by the same rule-based algorithm (calculationService.js) that we're trying to approximate. The ML model learns to predict the output of the financial algorithm from the inputs, without needing real-world labelled data which would be impossible to collect at scale with privacy concerns. The synthetic profiles follow realistic Indian income distributions, so the model generalises well to real inputs.

Q9. What are the 16 features your XGBoost model uses?

8 raw inputs: income, expenses, EMI, savings, dependents, age, has_insurance (binary), renting (binary). Plus 8 derived features: job_enc (label-encoded job type), life_enc (life stage), city_enc (city tier), emi_ratio (EMI÷income), expense_ratio (expenses÷income), surplus (income−expenses−EMI), savings_ratio (savings÷annual_income), survival_months (savings÷expenses). The derived features help the model capture relationships the raw numbers don't directly express.

Q10. What do R^2=0.949 and Accuracy=85.3% mean?

R^2=0.949 means the regression model (risk score predictor) explains 94.9% of the variance in risk scores — very high. An R^2 of 1.0 would be perfect. Accuracy=85.3% means the classifier correctly predicts the risk level (Low/Medium/High/Critical) 85.3% of the time on unseen test data. The MAE of 4.22 means on average the predicted score is within ±4.22 points of the actual rule-based score.

AI / LLM Integration

Q11. What is Groq and why did you use it over OpenAI?

Groq is an AI inference platform using custom LPU (Language Processing Unit) chips that run LLMs at ~500 tokens/second — roughly 10× faster than typical GPU-based inference. We chose Groq because: it's free tier supports our demo use case, it has an OpenAI-compatible API (easy integration), LLaMA-3.3-70b is open-source (no licensing issues), and the speed means our API responds in 2–3 seconds rather than 10–15.

Q12. Why is the temperature set to 0.35 for analysis but higher for the chatbot?

Temperature controls randomness. For financial analysis (aiService.js), we want deterministic, factual output — a temperature of 0.35 means the model gives consistent, reliable advice rather than hallucinating creative answers. The chatbot is more conversational so slightly higher temperature makes responses feel more natural and varied. Low temperature = more focused, high temperature = more creative.

Frontend & UX

Q13. How does the dark/light theme toggle work?

React Context API (ThemeCtx) holds the dark boolean state in the root App.jsx. A custom CSS variable system switches between two sets of color tokens (--bg, --surface, --text etc.) based on a data-theme attribute on the root div. All components access the theme via the useTheme() hook. Framer Motion transition handles smooth 0.4s animations between themes. The preference is stored in React state (not localStorage) so resets on refresh.

Q14. Why did you use jsPDF for PDF generation on the client side?

Client-side PDF generation (jsPDF in the browser) means: no extra server load, immediate download without a round-trip to the server, and the user's data never needs to be sent again just for a PDF. The alternative — server-side PDF generation with Puppeteer or pdfkit — would add complexity and latency. jsPDF v4 supports modern JavaScript, React integration, and canvas drawing for charts.

Q15. Explain the voice input/output feature.

Voice input uses the browser's Web Speech API — specifically SpeechRecognition with lang='en-IN' for Indian English. When the mic button is pressed, it starts continuous recognition and appends recognised text to the input field. Voice output uses SpeechSynthesis API — when the AI sends a response, speechSynthesis.speak() is called with the text. An animated sound-wave SVG shows while speaking. This is entirely browser-native — no external API needed.

Architecture & Design

Q16. Why did you separate the ML service from the Node.js backend?

Python has the best ML ecosystem (scikit-learn, XGBoost, numpy, pandas) while Node.js is better for web APIs. Running the ML model in Node would require awkward Python-in-Node bridges (python-shell) or rewriting XGBoost in JavaScript. The microservice pattern lets each service use its ideal runtime, scale independently, and be replaced or updated without touching the main backend. The Node server calls Flask via HTTP with a 4-second timeout.

Q17. How does the admin panel work? How is the first admin created?

The first user who registers automatically gets the 'admin' role (authController.js line: role: users.length === 0 ? 'admin' : 'user'). Admin routes (/api/admin/*) are protected by two middleware layers: verifyToken (checks valid JWT) then requireAdmin (checks role === 'admin'). The admin panel shows total users, total analyses, today's count, risk distribution, average fund target, and allows changing user roles.

Q18. What happens when a user submits the form? Walk through the complete flow.

1. CalculatorForm.jsx collects 11 inputs across 6 steps and validates them. 2. On submit, it calls POST /api/analyze. 3. AnalysisPage.jsx shows a animated "boot sequence" while waiting. 4. Express validates inputs (422 if invalid), normalises field names, runs the calculation engine synchronously. 5. Groq AI and XGBoost ML predictions run in parallel via Promise.all(). 6. A unified JSON response is built combining all three results. 7. React receives the response, stores it in state, navigates to Dashboard. 8. The user optionally saves to history (POST /api/save).

Deployment & Environment

Q19. What environment variables are needed to run this project?

Server needs: GROQ_API_KEY (for LLM features), MONGO_URI (for MongoDB, optional), JWT_SECRET (for auth security), SMTP_USER and SMTP_PASS (for email). ML service needs no special env vars — it uses local model files. Client needs the API URL (default localhost:5001). Without GROQ_API_KEY, the app uses rule-based fallback. Without MONGO_URI, it uses JSON files. So technically the app runs with zero environment variables configured.

Q20. What are the three ports used and what runs on each?

> Port 5173 — Vite dev server (React frontend). Port 5001 — Express API server (Node.js backend). Port 5002 — Flask ML service (Python XGBoost). In production, Vite would be built to static files and served via Nginx/CDN, while only ports 5001 and 5002 would be running as backend services.

13. Key Code Walkthroughs

SECTION 13

Parallel AI + ML Execution (analyzeController.js)

```
// Run Groq AI insights and XGBoost ML prediction IN PARALLEL
const [aiResult, mlResult] = await Promise.all([
  aiService.generateAIInsights(calcResult, body),  // calls Groq API (8s timeout)
  mlService.getMLPrediction(body),                 // calls Flask ML (4s timeout)
]);

// Agreement: do the rule-based and ML models agree on risk level?
const mlAgreement = mlResult.source === 'xgboost'
  ? mlResult.mlRiskLevel === calcResult.riskLevel
    ? 'agree'
    : 'disagree'
  : 'unavailable';
```

Risk Score Calculation — EMI Factor (calculationService.js)

```
/* EMI-to-income ratio: RBI recommends below 30% */
const emiRatio = income > 0 ? emiAmt / income : 0;

if (emiRatio > 0.5) {
  monthsRecommended += 2.5; riskScore += 32;          // critical debt load
} else if (emiRatio > 0.3) {
  monthsRecommended += 1.5; riskScore += 16;          // moderate burden
} else if (emiRatio > 0) {
  monthsRecommended += 0.5; riskScore += 6;           // minor debt
} else {
  protectiveFactors.push({ factor: 'Zero EMI / Debt-Free' });  // good!
}
```

JWT Token Creation & Verification

```
// Create token on login/register (authController.js)
function signToken(user) {
  return jwt.sign(
    { id: user.id, email: user.email, name: user.name, role: user.role },
    JWT_SECRET,
    { expiresIn: '7d' }
  );
```

```
}

// Verify on each request (middleware/auth.js)
req.user = jwt.verify(token, JWT_SECRET);  // throws if invalid or expired
```

XGBoost Feature Vector Construction (serve.py)

```
def build_feature_vector(data):
    # Raw features
    income    = max(0, float(data.get("monthlyIncome", 0)))
    emi       = max(0, float(data.get("emi", 0)))

    # Derived ratios — these help the model learn non-linear relationships
    emi_ratio       = emi / income if income > 0 else 0
    expense_ratio   = expenses / income if income > 0 else 0
    survival_months = savings / expenses if expenses > 0 else 0

    # Categorical encoding (label encoding, not one-hot)
    job_enc = JOB_CLASSES.index(job)  # "corporate" → 1, "govt" → 4, etc.

    return [income, expenses, emi, savings, deps, age,
            insured, renting,
            job_enc, life_enc, city_enc,
            emi_ratio, expense_ratio, surplus, savings_ratio, survival_months]
```

MongoDB Connection with Fallback (db.js)

```
const connectDB = async () => {
  const uri = process.env.MONGO_URI;
  if (!uri) {
    logger.warn('MONGO_URI not set — using file-based storage');
    return false;  // graceful fallback to data.json
  }
  try {
    await mongoose.connect(uri, {
      serverSelectionTimeoutMS: 5000,  // fail fast if unreachable
      maxPoolSize: 10,                 // connection pool size
    });
    // Register lifecycle events for auto-reconnect
    mongoose.connection.on('disconnected', () => logger.warn('MongoDB disconnected'));
    return true;
  } catch (err) {
    logger.error('MongoDB failed — file fallback active', { error: err.message });
    return false;
```

```
  }
};
```

14. Quick Reference — Numbers & Facts

SECTION 14

ML Model Facts

METRIC	VALUE
Training Algorithm	XGBoost (Gradient Boosting)
Training Samples	6,400 (80/20 split from 8,000)
Test Samples	1,600
Features Used	16 (8 raw + 8 derived)
n_estimators	400
max_depth	7
Regressor R^2	0.9486 (94.86%)
Classifier Accuracy	0.8531 (85.31%)
MAE (Mean Abs Error)	4.22 risk points

System Facts

ITEM	VALUE
Frontend Port	5173 (Vite)
Backend Port	5001 (Express)
ML Service Port	5002 (Flask)
JWT Expiry	7 days
bcrypt Cost Factor	12
Groq Timeout	8,000ms (AI insights)
ML Service Timeout	4,000ms
Chatbot Timeout	12,000ms
Max Request Body	10KB
Max History Records	100 (file mode)
Chat History Window	Last 8 turns
LLM Temperature	0.35 (analysis), ~0.7 (chat)

Risk Thresholds Quick Reference

RISK LEVEL	SCORE	MEANING

JOB TYPE	RISK POINTS	EXTRA MONTHS
Government	+5	+0 months

Low	0–25	Well-prepared, stable income, insured
Medium	26–50	Partially funded, manageable risks
High	51–72	Under-funded, volatile income
Critical	73–100	Dangerously exposed, urgent action needed

Corporate	+22	+1.5 months
Business Owner	+38	+3.5 months
Gig Worker	+42	+3.5 months
Freelancer	+48	+4.5 months

How to Run the Project

```
# Terminal 1 — Start Backend (Node/Express)
cd server
npm install
node index.js          # runs on http://localhost:5001

# Terminal 2 — Start ML Service (Python/Flask)
cd ml
pip install flask flask-cors xgboost scikit-learn pandas numpy joblib
python train_model.py  # first time: generates model files in ml/model/
python serve.py        # runs on http://localhost:5002

# Terminal 3 — Start Frontend (React/Vite)
cd client
npm install
npm run dev            # runs on http://localhost:5173
```

✨ 15. What Makes This Project Unique?

SECTION 15

1. Triple-Engine Analysis

Most financial calculators use only one algorithm. SafetyNet.ai runs a rule-based financial engine, a generative AI (LLaMA), and a trained ML model (XGBoost) — then compares their results to build user trust through the **AI/ML Agreement Indicator**.

2. India-First Design

Built specifically for India — uses ₹ INR, understands Indian city tiers (Tier 1/2/3), Indian job types (govt/PSU vs gig), Indian financial products (PPF, ELSS, NPS, SIP), and references RBI and SEBI benchmarks.

3. Graceful Degradation

Three independent fallback layers ensure the app always works — even with no internet (fallback insights), no database (JSON files), or no ML service (skips ML section). Zero hard dependencies for core functionality.

4. Voice-Enabled Chatbot

The AI chatbot supports both voice input (en-IN speech recognition) and voice output (TTS auto-reads responses) — making it accessible and unique among financial tools for Indian users.

5. Stress Testing Feature

Users can simulate worst-case scenarios — income drops and expense spikes — to see how long their emergency fund would last. This "what if" planning tool goes beyond basic calculators.

6. Real-Time PDF Report

Client-side PDF generation using jsPDF creates a professional 2-page A4 report with risk gauge, metrics, AI insights, and investment plan — instantly downloaded with no server round-trip.

🏆 **Summary for Viva:** This project demonstrates mastery of full-stack development (React + Node + Python), AI/ML integration (LLM + supervised learning), secure authentication (JWT + bcrypt), database design (MongoDB with file fallback), microservice architecture, and production-quality UX patterns (animations, voice, dark mode, PDF export).